

INFOB2DA | Practical Assignment 4 | Si-8

Irvan Ilahibaks 8081847 - Timon Baris 2173182 - Waling van Dongen 1176226

TA: Sita Newer Date: 27-10-2024

Task 1: Create an interactive dashboard (Level 2)

```
1 import pandas as pd
2
3 dfData = pd.read_csv('airlinedelaycauses_DelayedFlights.csv')
```

```
1 dfData.info() |
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1936758 entries, 0 to 1936757
Data columns (total 30 columns):
#   Column              Dtype
---  -
0   Unnamed: 0          int64
1   Year                int64
2   Month              int64
3   DayOfMonth         int64
4   DayOfWeek          int64
5   DepTime            float64
6   CRSDepTime         int64
7   ArrTime            float64
8   CRSArrTime         int64
9   UniqueCarrier      object
10  FlightNum          int64
11  TailNum            object
12  ActualElapsedTime  float64
13  CRSElapsedTime     float64
14  AirTime            float64
15  ArrDelay           float64
16  DepDelay           float64
17  Origin             object
18  Dest              object
19  Distance           int64
20  TaxiIn            float64
21  TaxiOut           float64
22  Cancelled          int64
23  CancellationCode   object
24  Diverted           int64
25  CarrierDelay       float64
26  WeatherDelay       float64
27  NASDelay           float64
28  SecurityDelay      float64
29  LateAircraftDelay  float64
dtypes: float64(14), int64(11), object(5)
memory usage: 443.3+ MB
```

Summary statistics																		
Year	Month	DayofMonth	DayOfWeek	DepTime	CRSDepTime	ArrTime	CRSArrTime	FlightNum	...	Distance	TaxiIn	TaxiOut	Cancelled	Diverted	CarrierDelay	WeatherDelay		
758.0	1.936758e+06	1.936758e+06	1.936758e+06	1.936758e+06	1.936758e+06	1.929648e+06	1.936758e+06	1.936758e+06	...	1.936758e+06	1.929648e+06	1.936303e+06	1.936758e+06	1.936758e+06	1.247488e+06	1.247488e+06		
008.0	6.111106e+00	1.575347e+01	3.984827e+00	1.518534e+03	1.467473e+03	1.610141e+03	1.634225e+03	2.184263e+03	...	7.656862e+02	6.812975e+00	1.823220e+01	3.268348e-04	4.003598e-03	1.917940e+01	3.703571e+00		
0.0	3.482546e+00	8.776272e+00	1.995966e+00	4.504853e+02	4.247668e+02	5.481781e+02	4.646347e+02	1.944702e+03	...	5.744797e+02	5.273595e+00	1.433853e+01	1.807562e-02	6.314722e-02	4.354621e+01	2.149290e+01		
008.0	1.000000e+00	1.000000e+00	1.000000e+00	1.000000e+00	0.000000e+00	1.000000e+00	0.000000e+00	1.000000e+00	...	1.100000e+01	0.000000e+00	0.000000e+00	0.000000e+00	0.000000e+00	0.000000e+00	0.000000e+00		
008.0	3.000000e+00	8.000000e+00	2.000000e+00	1.203000e+03	1.135000e+03	1.316000e+03	1.325000e+03	6.100000e+02	...	3.380000e+02	4.000000e+00	1.000000e+01	0.000000e+00	0.000000e+00	0.000000e+00	0.000000e+00		
008.0	6.000000e+00	1.600000e+01	4.000000e+00	1.545000e+03	1.510000e+03	1.715000e+03	1.705000e+03	1.543000e+03	...	6.060000e+02	6.000000e+00	1.400000e+01	0.000000e+00	0.000000e+00	2.000000e+00	0.000000e+00		
008.0	9.000000e+00	2.300000e+01	6.000000e+00	1.900000e+03	1.815000e+03	2.030000e+03	2.014000e+03	3.422000e+03	...	9.980000e+02	8.000000e+00	2.100000e+01	0.000000e+00	0.000000e+00	2.100000e+01	0.000000e+00		
008.0	1.200000e+01	3.100000e+01	7.000000e+00	2.400000e+03	2.359000e+03	2.400000e+03	2.400000e+03	9.742000e+03	...	4.962000e+03	2.400000e+02	4.220000e+02	1.000000e+00	1.000000e+00	2.436000e+03	1.352000e+03		

```

1 print(dfData.to_string)

<bound method DataFrame.to_string of
0      0  2008      1      3      4  2003.0  1955
1      1  2008      1      3      4   754.0   735
2      2  2008      1      3      4   628.0   620
3      4  2008      1      3      4  1829.0  1755
4      5  2008      1      3      4  1940.0  1915
...
1936753  7009710  2008     12     13      6  1250.0  1220
1936754  7009717  2008     12     13      6   657.0   600
1936755  7009718  2008     12     13      6  1007.0   847
1936756  7009726  2008     12     13      6  1251.0  1240
1936757  7009727  2008     12     13      6  1110.0  1103

ArrTime  CRSArrTime  UniqueCarrier  ...  TaxiIn  TaxiOut  Cancelled \
0      2211.0      2225           WN  ...    4.0    8.0         0
1      1002.0      1000           WN  ...    5.0   10.0         0
2       804.0       750           WN  ...    3.0   17.0         0
3      1959.0      1925           WN  ...    3.0   10.0         0
4      2121.0      2110           WN  ...    4.0   10.0         0
...
1936753    1617.0     1552           DL  ...    9.0   18.0         0
1936754     904.0       749           DL  ...   15.0   34.0         0
1936755    1149.0     1010           DL  ...    8.0   32.0         0
1936756    1446.0     1437           DL  ...   13.0   13.0         0
1936757    1413.0     1418           DL  ...    8.0   11.0         0

CancellationCode  Diverted  CarrierDelay  WeatherDelay  NASDelay \
0                N         0           NaN           NaN           NaN
1                N         0           NaN           NaN           NaN
2                N         0           NaN           NaN           NaN
3                N         0            2.0           0.0           0.0
4                N         0           NaN           NaN           NaN
...
1936753          N         0            3.0           0.0           0.0
1936754          N         0            0.0           57.0           18.0
1936755          N         0            1.0           0.0           19.0
1936756          N         0           NaN           NaN           NaN
1936757          N         0           NaN           NaN           NaN

SecurityDelay  LateAircraftDelay
0            NaN                NaN
1            NaN                NaN
2            NaN                NaN
3            0.0               32.0
4            NaN                NaN
...
1936753        0.0               22.0
1936754        0.0                0.0
1936755        0.0               79.0
1936756        NaN                NaN
1936757        NaN                NaN

```

The focus of this analysis is to determine whether certain factors affect flight delays, specifically looking into the likelihood of delays based on various recorded metrics. Initially, we analysed information from the entire dataset, which includes data on over 500,000 flights. Observing the delay status, we noticed that most flights belonged to the "non-delayed" category, indicating that most flights departed close to their scheduled time. However, a significant portion of flights were indeed delayed.

```

1 # Count the amount of NaN values of each column
2 nan_counts = dfData.isna().sum()
3
4 print("Number of NaN values in each column:")
5 print(nan_counts)

```

Number of NaN values in each column:

```

Unnamed: 0      0
Year            0
Month           0
DayofMonth      0
DayOfWeek       0
DepTime         0
CRSDepTime      0
ArrTime        7110
CRSArrTime      0
UniqueCarrier   0
FlightNum       0
TailNum         5
ActualElapsedTime 8387
CRSElapsedTime  198
AirTime         8387
ArrDelay        8387
DepDelay        0
Origin          0
Dest            0
Distance        0
TaxiIn          7110
TaxiOut         455
Cancelled       0
CancellationCode 0
Diverted        0
CarrierDelay    689270
WeatherDelay    689270
NASDelay        689270
SecurityDelay   689270
LateAircraftDelay 689270
CRSDepTimeMinutes 0
DepTimeMinutes  0
DepartureDifference 0
dtype: int64

```

We also noticed that there were a lot of NaN values but decided that we did not need to remove them. The first reason was that the amount of NaN values was too big, when removing the rows with NaN values it would change the dataset too much. Also a lot of the NaN values are meant to be calculated to show that there were cases where they were no delay. This should also be encountered for when looking at the data.

Firstly, we analysed the dataset to see if there were any interesting findings we could show in a plot. We looked at all the column names in a list to make it easier to look at certain correlating columns.

```
dfData.columns #show the different columns in the data set
```

```

Index(['Unnamed: 0', 'Year', 'Month', 'DayofMonth', 'DayOfWeek', 'DepTime',
      'CRSDepTime', 'ArrTime', 'CRSArrTime', 'UniqueCarrier', 'FlightNum',
      'TailNum', 'ActualElapsedTime', 'CRSElapsedTime', 'AirTime', 'ArrDelay',
      'DepDelay', 'Origin', 'Dest', 'Distance', 'TaxiIn', 'TaxiOut',
      'Cancelled', 'CancellationCode', 'Diverted', 'CarrierDelay',
      'WeatherDelay', 'NASDelay', 'SecurityDelay', 'LateAircraftDelay'],
      dtype='object')

```

```
import matplotlib.pyplot as plt
#filter for canceled flights and group by month to count cancellations
AmountCancellationsEachMonth= dfData[dfData['Cancelled'] == 1].groupby('Month').size()
AmountNonCancellationsEachMonth= dfData[dfData['Cancelled'] == 0].groupby('Month').size()
print("Amount of continued flights each month:",AmountNonCancellationsEachMonth,
      "\nAmount of Cancelled flights each month:",AmountCancellationsEachMonth)
```

Amount of continued flights each month: Month

```
1    183527
2    189534
3    200842
4    155264
5    153072
6    200914
7    182945
8    162648
9     95539
10   103466
11   105469
12   202905
dtype: int64
```

Amount of Cancelled flights each month: Month

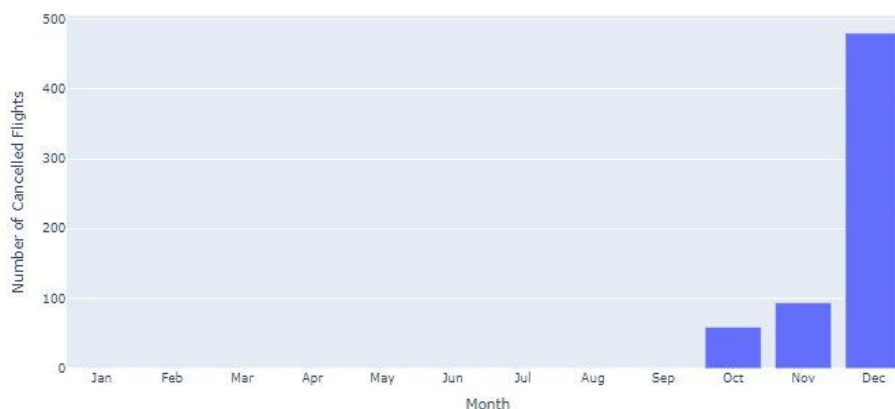
```
10     59
11     94
12    480
dtype: int64
```

```
#filter to look at the months where flights were cancelled
MonthsWithCancellations = dfData.loc[dfData['Cancelled'] == 1, 'Month'].unique()
print(MonthsWithCancellations)
```

```
[10 11 12]
```

```
1 import plotly.express as px
2 import pandas as pd
3
4 #define MonthRange from 1 to 12 to ensure all months are included
5 MonthRange = range(1, 13)
6
7 #reindex AmountCancellationsEachMonth to ensure all months are present, filling missing months with 0
8 AmountCancellationsEachMonth = AmountCancellationsEachMonth.reindex(MonthRange, fill_value=0)
9
10 #plot the data
11 fig = px.bar(x=list(MonthRange), y=AmountCancellationsEachMonth.values,
12             labels={'x': 'Month', 'y': 'Number of Cancelled Flights'},
13             title='Number of Canceled Flights Each Month')
14
15 fig.update_layout(
16     xaxis=dict(
17         title='Month',
18         tickmode='array',
19         tickvals=list(MonthRange),
20         ticktext=['Jan', 'Feb', 'Mar', 'Apr', 'May', 'Jun', 'Jul', 'Aug', 'Sep', 'Oct', 'Nov', 'Dec']),
21     yaxis=dict(title='Number of Cancelled Flights'),
22 )
23
24 #show the figure
25 fig.show()
26
```

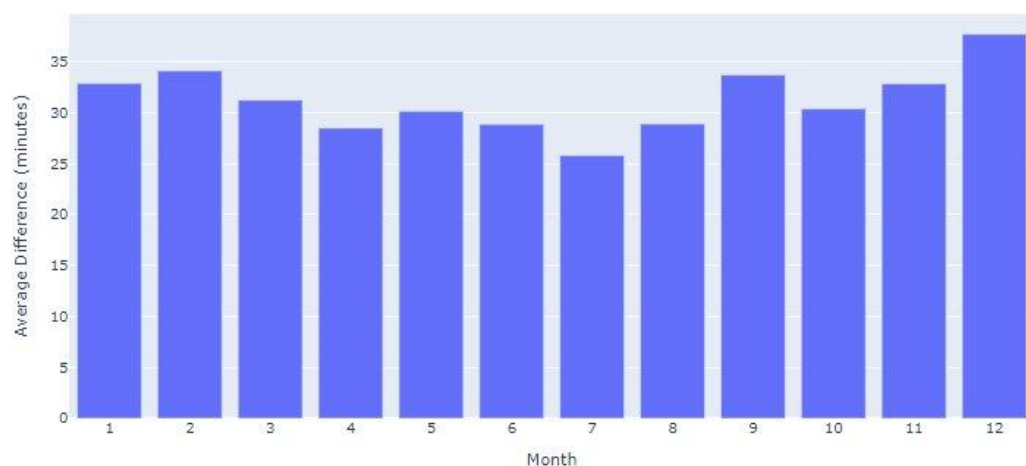
Number of Canceled Flights Each Month



Monthly cancellations bar chart: We created a bar chart showing the number of cancelled flights for each month. This chart reveals seasonal trends, with certain months experiencing significantly higher cancellation rates. For instance, winter months might have higher cancellations, possibly due to weather-related issues, while summer months show fewer cancellations. This chart provides insight into when cancellations are most frequent, helping airlines anticipate and prepare for peak cancellation periods.

```
1 import plotly.express as px
2
3 #function to convert the values into minutes
4 def ConvertInMinutes(time):
5     hours = time // 100
6     minutes = time % 100
7     return hours * 60 + minutes
8
9 #convert the time into minutes
10 dfData['CRSDepTimeMinutes'] = dfData['CRSDepTime'].apply(ConvertInMinutes)
11 dfData['DepTimeMinutes'] = dfData['DepTime'].apply(ConvertInMinutes)
12
13 #calculate the difference
14 dfData['DepartureDifference'] = dfData['DepTimeMinutes'] - dfData['CRSDepTimeMinutes']
15
16 #group by month and calculate the average difference
17 AverageDifference = dfData.groupby('Month')['DepartureDifference'].mean().reset_index()
18
19 #create the bar chart
20 fig = px.bar(AverageDifference, x='Month', y='DepartureDifference',
21             title='Average Difference Between Scheduled and Actual Departure Time',
22             labels={'DepartureDifference': 'Average Difference (minutes)', 'Month': 'Month'})
23
24 fig.update_layout(
25     xaxis=dict(tickvals=[1, 2,3,4,5,6,7,8,9,10,11,12]),
26     bargap=0.2
27 )
28
29 #show the figure
30 fig.show()
```

Average Difference Between Scheduled and Actual Departure Time



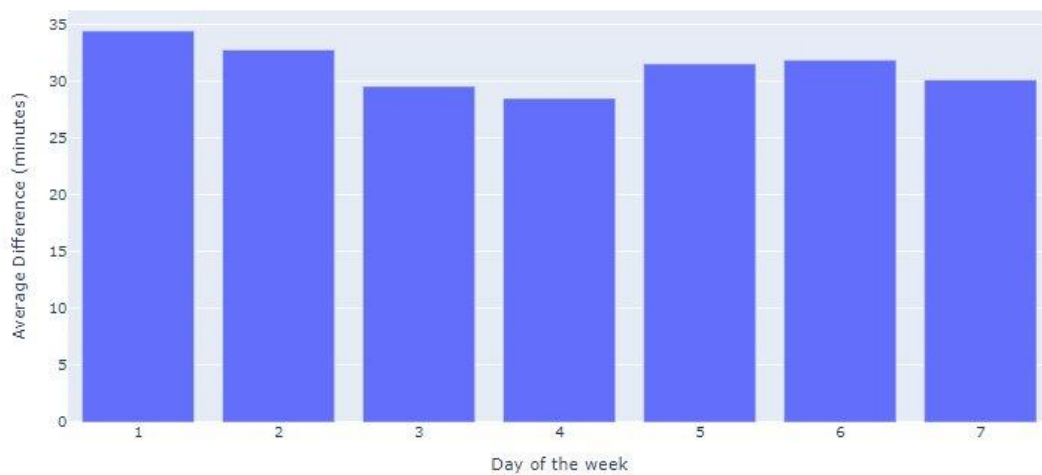
This bar chart visualizes the **Average difference between scheduled and actual departure time** for each month. By plotting the average delay per month, we gain a clear overview of how departure delays vary throughout the year.

```

1 import plotly.express as px
2
3 #convert the time into minutes
4 dfData['CRSDepTimeMinutes'] = dfData['CRSDepTime'].apply(ConvertInMinutes)
5 dfData['DepTimeMinutes'] = dfData['DepTime'].apply(ConvertInMinutes)
6
7 #calculate the difference
8 dfData['DepartureDifference'] = dfData['DepTimeMinutes'] - dfData['CRSDepTimeMinutes']
9
10 #group by Month and calculate the average difference
11 AverageDifference = dfData.groupby('DayOfWeek')['DepartureDifference'].mean().reset_index()
12
13 #create the bar chart
14 fig = px.bar(AverageDifference, x='DayOfWeek', y='DepartureDifference',
15             title='Average Difference Between Scheduled and Actual Departure Time',
16             labels={'DepartureDifference': 'Average Difference (minutes)', 'DayOfWeek': 'Day of the week'})
17
18 #show the figure
19 fig.show()
20

```

Average Difference Between Scheduled and Actual Departure Time



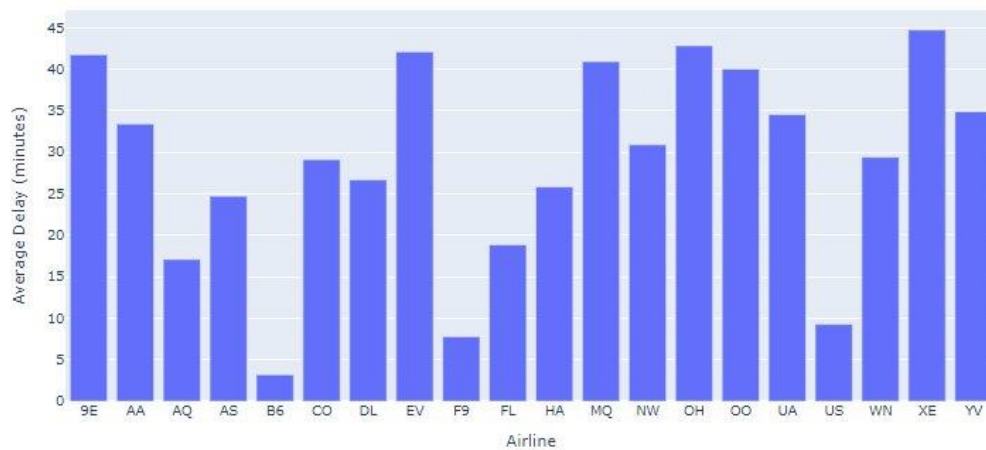
Average delay by day of the week: Another bar chart visualizes the average delay across each day of the week, indicating patterns in delays by weekday. We observed that some days, such as Mondays or Fridays, might have higher average delays, suggesting that end-of-week or start-of-week travel could be more prone to delays. This could be due to increased travel volume or operational resets, offering valuable insights for scheduling improvements.


```

1 import plotly.express as px
2
3 #group by 'UniqueCarrier' and calculate the average delay
4 AverageDelayByAirline = dfData.groupby('UniqueCarrier')['DepartureDifference'].mean().reset_index()
5
6 #create bar chart for average delay by airline
7 fig = px.bar(AverageDelayByAirline, x='UniqueCarrier', y='DepartureDifference',
8             title='Average Departure Delay by Airline',
9             labels={'DepartureDifference': 'Average Delay (minutes)', 'UniqueCarrier': 'Airline'})
10
11 fig.update_layout(bargap=0.2)
12
13 #show the figure
14 fig.show()
15

```

Average Departure Delay by Airline



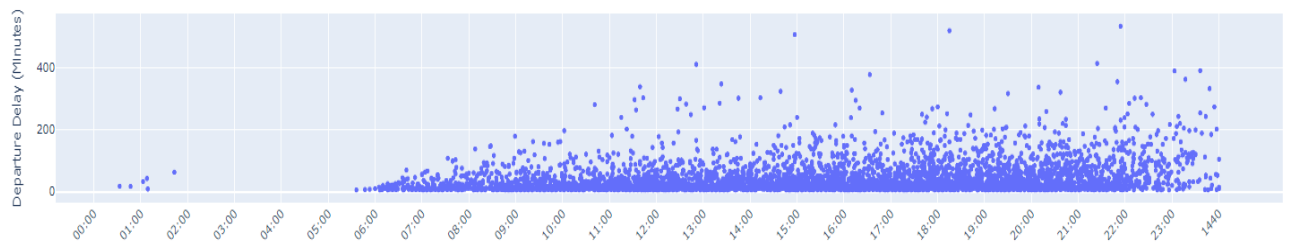
This bar chart presents the **Average departure delay by airline**, offering a focused view on how each airline performs in terms of timely departures.

```

1 import pandas as pd
2 import plotly.express as px
3
4 def ConvertToMinutes(time):
5     hours = time // 100
6     minutes = time % 100
7     return hours * 60 + minutes
8
9 #filter for delayed flights and create a DataFrame with valid 'DepTime' entries
10 df_delays = dfData[dfData['DepartureDifference'] > 0][['DepTime', 'DepartureDifference']].copy()
11 df_delays['DepTime'] = pd.to_numeric(df_delays['DepTime'], errors='coerce') # Convert to numeric and handle errors
12 df_delays.dropna(subset=['DepTime'], inplace=True) # Drop rows with NaN values
13
14 #convert valid DepTime to minutes and format it as HH:MM
15 df_delays['DepTimeMinutes'] = df_delays['DepTime'].apply(ConvertToMinutes)
16 df_delays['DepTimeFormatted'] = df_delays['DepTime'].apply(lambda x: f"{int(x // 100):02d}:{int(x % 100):02d}")
17
18 #visualize a sample of df_delays
19 fig = px.scatter(
20     df_delays.sample(n=5000, random_state=1), # Use a sample for efficiency
21     x='DepTimeMinutes', # Use minutes for proper sorting
22     y='DepartureDifference',
23     title='Departure Delays by Time of Day',
24     labels={'DepTimeMinutes': 'Departure Time (HH:MM)', 'DepartureDifference': 'Delay (minutes)'}
25 )
26
27 #update x-axis ticks to show HH:MM format
28 fig.update_traces(marker=dict(size=5))
29 fig.update_layout(
30     xaxis=dict(
31         title='Departure Time (HH:MM)',
32         tickvals=list(range(0, 1441, 60)), #every hour (60 minutes)
33         ticktext=[f"{h:02d}:00" for h in range(24)] #tick labels from 00:00 to 23:00
34     ),
35     yaxis=dict(title='Departure Delay (Minutes)'),
36     xaxis_tickangle=-45 #tilt x-axis labels for better visibility
37 )
38
39 #show the figure
40 fig.show()
41

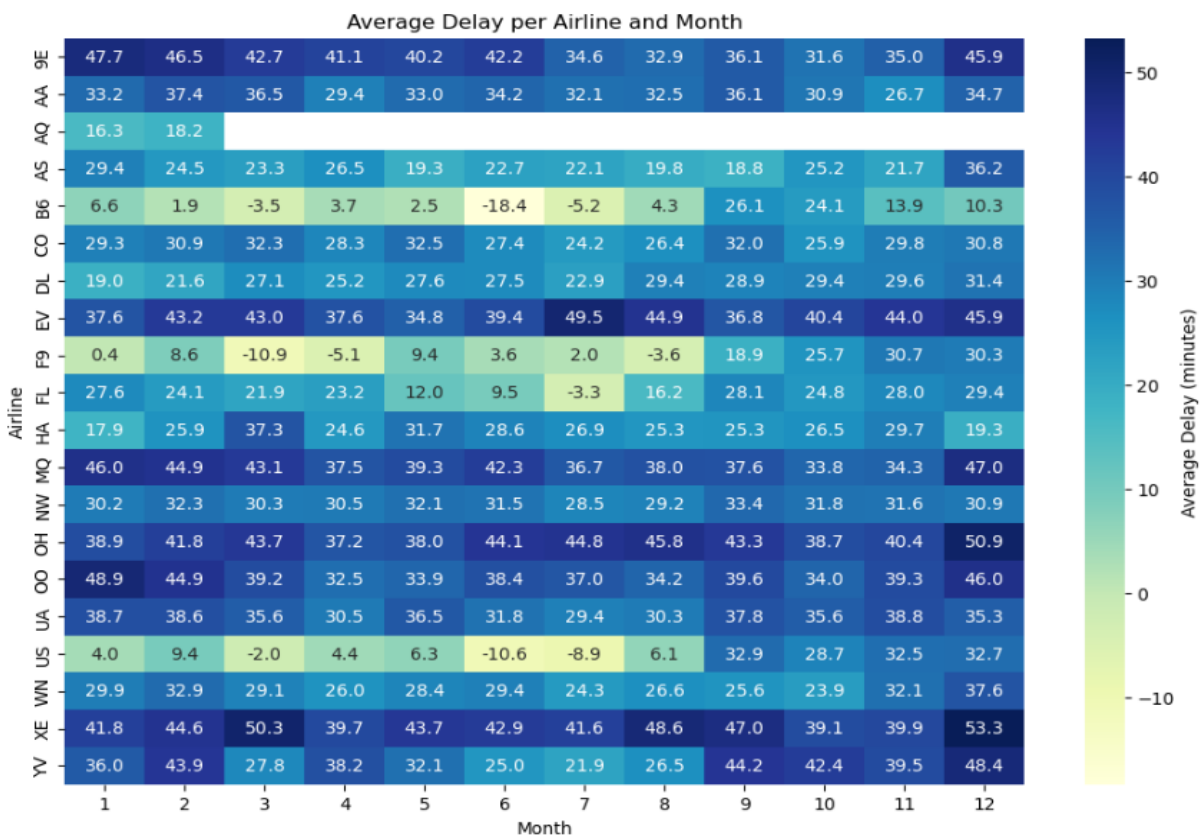
```

Departure Delays by Time of Day



Departure delays by time of day (scatter plot): Finally, a scatter plot of departure delays across different times of day helped us identify time-based patterns. There was a noticeable clustering of delays during certain peak hours, such as early morning or late afternoon.

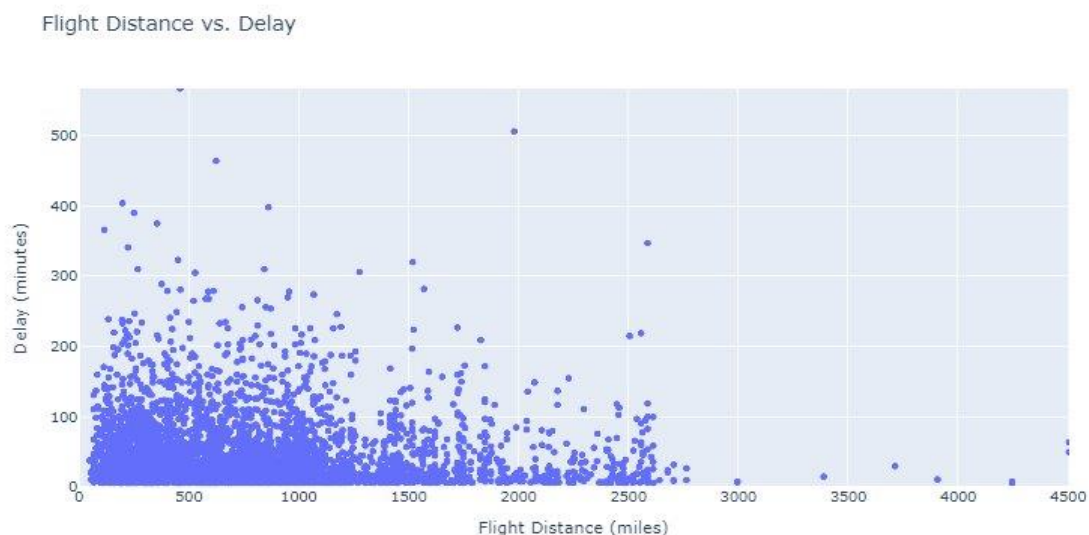
```
1 import seaborn as sns
2 import matplotlib.pyplot as plt
3
4 #bereken de gemiddelde vertraging per Luchtvaartmaatschappij en maand
5 heatmap_data = dfData.groupby(['UniqueCarrier', 'Month'])['DepartureDifference'].mean().unstack()
6
7 plt.figure(figsize=(12, 8))
8 sns.heatmap(heatmap_data, annot=True, fmt=".1f", cmap="YlGnBu", cbar_kws={'label': 'Gemiddelde Vertraging (minuten)'})
9 plt.title('Gemiddelde Vertraging per Luchtvaartmaatschappij en Maand')
10 plt.xlabel('Maand')
11 plt.ylabel('Luchtvaartmaatschappij')
12 plt.show()
13
```



Average delay by airline and month (Heatmap): To explore seasonal and airline-specific delay trends, we created a heatmap showing the average delay for each airline

by month. This visualisation highlights which airlines tend to have higher delays in particular months. We noticed that certain airlines consistently experience delays during peak travel seasons, which could be due to operational or logistical factors. This chart is especially useful for identifying airlines with more significant seasonal fluctuations in delay.

```
1 import plotly.express as px
2
3 #sample 5000 rows from the dataset for performance
4 sample_data = dfData.sample(n=5000, random_state=1)
5
6 fig = px.scatter(
7     sample_data,
8     x='Distance',
9     y='DepartureDifference',
10    title="Flight Distance vs. Delay",
11    labels={'Distance': 'Flight Distance (miles)', 'DepartureDifference': 'Delay (minutes)'}
12 )
13
14 # Update Layout to set the range for both axes
15 fig.update_layout(
16     xaxis=dict(range=[0, sample_data['Distance'].max()], title='Flight Distance (miles)'),
17     yaxis=dict(range=[0, sample_data['DepartureDifference'].max()], title='Delay (minutes)')
18 )
19
20 fig.show()
```



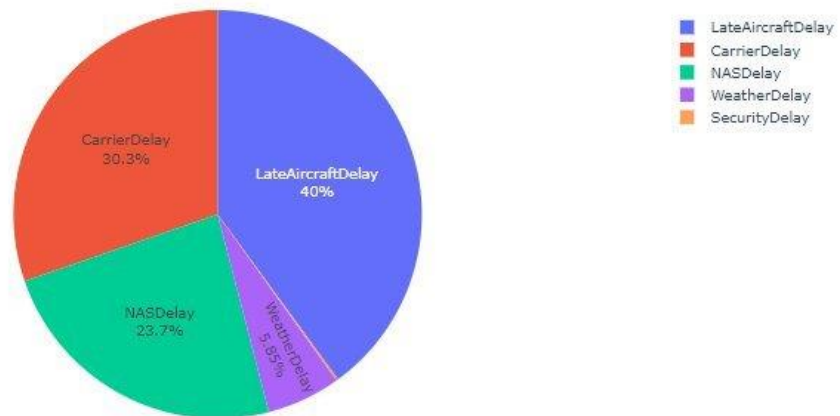
The scatter plot shows the relationship between how far flights travel and the delays they experience. By sampling 5,000 rows from the dataset, we've ensured that the visualisation remains efficient while still providing a solid overview of trends. In this plot, the x-axis represents the flight distance in miles, while the y-axis indicates the corresponding delays in minutes. It's interesting to see if there's a correlation between longer flights and the likelihood of delays. For instance, we might expect that longer distances could lead to more complexities in flight operations, possibly resulting in longer delays.

```

1 #summing each delay cause across all flights
2 delay_causes = dfData[['CarrierDelay', 'WeatherDelay', 'NASDelay', 'SecurityDelay', 'LateAircraftDelay']].sum()
3
4 #plot
5 fig = px.pie(values=delay_causes, names=delay_causes.index, title="Distribution of Delay Causes")
6 fig.update_traces(textposition='inside', textinfo='percent+label')
7
8 #show figure
9 fig.show()

```

Distribution of Delay Causes

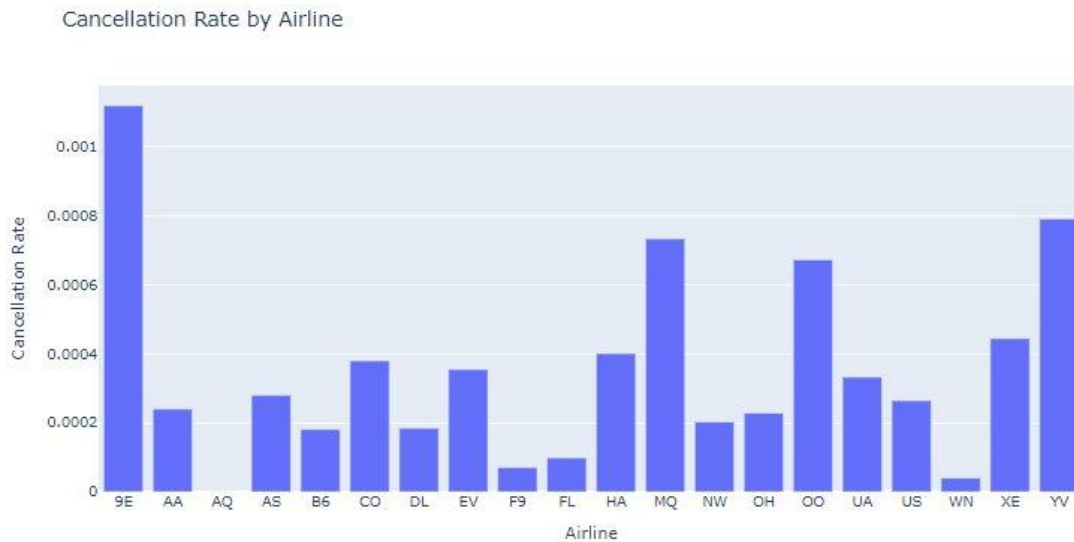


This pie chart illustrates the **distribution of delay causes**, providing a breakdown of the main reasons behind flight delays.

```

1 #calculate cancellation rate by airline
2 cancellation_rate = dfData.groupby('UniqueCarrier')['Cancelled'].mean().reset_index()
3
4 #plot
5 fig = px.bar(cancellation_rate, x='UniqueCarrier', y='Cancelled',
6             title='Cancellation Rate by Airline',
7             labels={'UniqueCarrier': 'Airline', 'Cancelled': 'Cancellation Rate'})
8 #show figure
9 fig.show()

```



The **cancellation rate by airline** bar chart presents an overview of the proportion of flights cancelled by each airline.

In summary, our analysis of flight delays and cancellations provides some interesting insights into how different airlines perform and the challenges they face. The various charts we've created show not just the average delays across different airlines and months but also the main reasons behind these delays. For example, the pie chart on delay causes really highlights how much factors like carrier delays and late arrivals impact on-time departures.

The average delay by airline gives us a chance to see which carriers are doing well and which ones might need to step up their game to keep flights on schedule. Plus, looking at the cancellation rates helps passengers understand which airlines are more likely to cancel flights, allowing them to make better choices when booking their travel.

Overall, these findings help everyone involved. Airlines, regulators, and travelers work toward making air travel more efficient and reliable. By figuring out these trends, we can aim to improve operational practices and enhance the travel experience, making flying a more convenient option for everyone.

Dashboards

```
1 import pandas as pd
2 import plotly.express as px
3
4 #Load your dataset
5 dfData = pd.read_csv('airlinedelaycauses_DelayedFlights.csv')
6
7 #function to convert the values into minutes
8 def ConvertInMinutes(time):
9     hours = time // 100
10    minutes = time % 100
11    return hours * 60 + minutes
12
13 #add minute conversions
14 dfData['CRSDepTimeMinutes'] = dfData['CRSDepTime'].apply(ConvertInMinutes)
15 dfData['DepTimeMinutes'] = dfData['DepTime'].apply(ConvertInMinutes)
16
17 #calculate the difference
18 dfData['DepartureDifference'] = dfData['DepTimeMinutes'] - dfData['CRSDepTimeMinutes']
19
20 ***1. Number of Canceled Flights Each Month**
21 AmountCancellationsEachMonth = dfData[dfData['Cancelled'] == 1].groupby('Month').size()
22 MonthRange = range(1, 13)
23 AmountCancellationsEachMonth = AmountCancellationsEachMonth.reindex(MonthRange, fill_value=0)
24
25 fig1 = px.bar(x=list(MonthRange), y=AmountCancellationsEachMonth.values,
26              labels={'x': 'Month', 'y': 'Number of Cancelled Flights'},
27              title='Number of Canceled Flights Each Month')
28
29 fig1.update_layout(
30     xaxis=dict(
31         title='Month',
32         tickmode='array',
33         tickvals=list(MonthRange),
34         ticktext=['Jan', 'Feb', 'Mar', 'Apr', 'May', 'Jun', 'Jul', 'Aug', 'Sep', 'Oct', 'Nov', 'Dec']),
35     yaxis=dict(title='Number of Cancelled Flights'),
36 )
37
38 fig1.show()
39
40 ***2. Average Difference Between Scheduled and Actual Departure Time (by Month)**
41 AverageDifferenceMonth = dfData.groupby('Month')['DepartureDifference'].mean().reset_index()
42
43 #convert MonthRange to a List
44 MonthRangeList = list(range(1, 13))
45
46 fig2 = px.bar(AverageDifferenceMonth, x='Month', y='DepartureDifference',
47              title='Average Difference Between Scheduled and Actual Departure Time',
48              labels={'DepartureDifference': 'Average Difference (minutes)', 'Month': 'Month'})
49
50 #update layout with correct tickvals
51 fig2.update_layout(
52     xaxis=dict(tickvals=MonthRangeList),
53     bargap=0.2
54 )
55
```

```

##3. Average Departure Delay by Day of the Week**
AverageDifferenceDay = dfData.groupby('DayOfWeek')['DepartureDifference'].mean().reset_index()
fig3 = px.bar(AverageDifferenceDay, x='DayOfWeek', y='DepartureDifference',
              title='Average Departure Delay by Day of the Week',
              labels={'DepartureDifference': 'Average Difference (minutes)', 'DayOfWeek': 'Day of the week'})

fig3.show()

##4. Average Departure Delay by Airline**
AverageDelayByAirline = dfData.groupby('UniqueCarrier')['DepartureDifference'].mean().reset_index()
fig4 = px.bar(AverageDelayByAirline, x='UniqueCarrier', y='DepartureDifference',
              title='Average Departure Delay by Airline',
              labels={'DepartureDifference': 'Average Delay (minutes)', 'UniqueCarrier': 'Airline'})

fig4.update_layout(bargap=0.2)
fig4.show()

##5. Departure Delays Distribution by Time of Day**
df_delays = dfData[dfData['DepartureDifference'] > 0][['DepTime', 'DepartureDifference']].copy()
df_delays['DepTime'] = pd.to_numeric(df_delays['DepTime'], errors='coerce') # Ensure numeric
df_delays.dropna(subset=['DepTime'], inplace=True) # Drop NaN values

#convert valid DepTime to minutes and format it as HH:MM
df_delays['DepTimeMinutes'] = df_delays['DepTime'].apply(ConvertInMinutes)
df_delays['DepTimeFormatted'] = df_delays['DepTime'].apply(lambda x: f"{int(x // 100):02d}:{int(x % 100):02d}")

fig5 = px.scatter(
    df_delays.sample(n=5000, random_state=1), # Sample for efficiency
    x='DepTimeMinutes', # Use minutes for proper sorting
    y='DepartureDifference',
    title='Departure Delays by Time of Day',
    labels={'DepTimeMinutes': 'Departure Time (HH:MM)', 'DepartureDifference': 'Delay (minutes)'}
)

#update x-axis ticks to show HH:MM format
fig5.update_traces(marker=dict(size=5))
fig5.update_layout(
    xaxis=dict(
        title='Departure Time (HH:MM)',
        tickvals=list(range(0, 1441, 60)), # Every hour (60 minutes)
        ticktext=[f"{h:02d}:{00}" for h in range(24)] # Tick labels from 00:00 to 23:00
    ),
    yaxis=dict(title='Departure Delay (Minutes)'),
    xaxis_tickangle=-45 # Tilt x-axis labels for better visibility
)

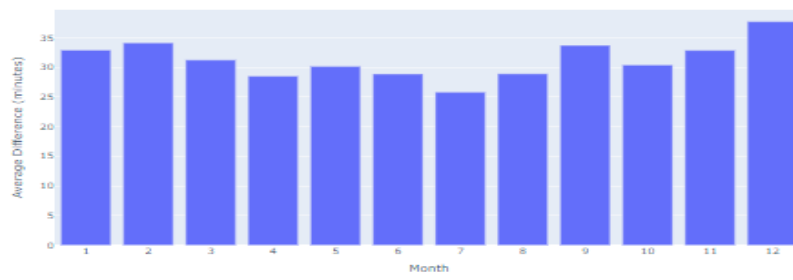
#show the figure
fig5.show()

```

Number of Canceled Flights Each Month



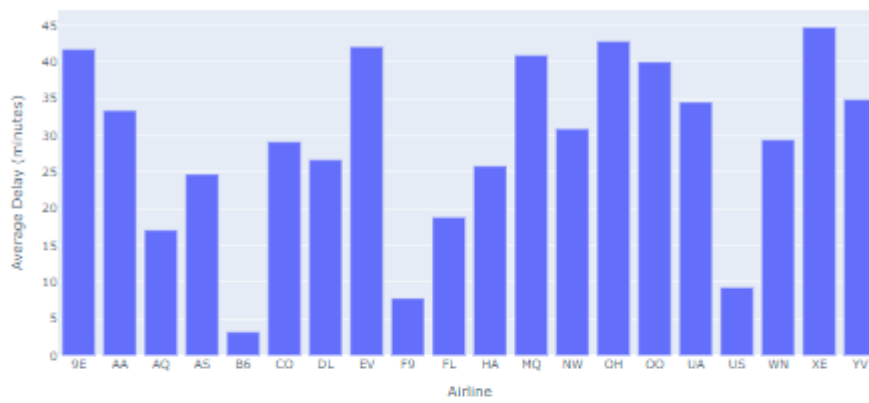
Average Difference Between Scheduled and Actual Departure Time



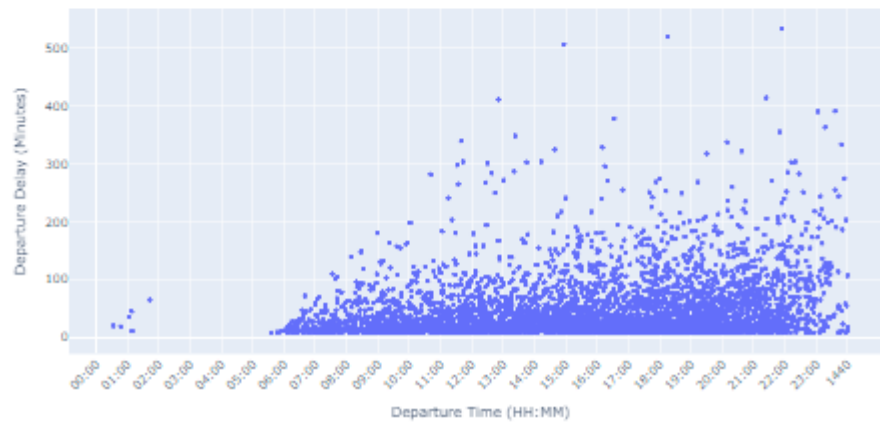
Average Departure Delay by Day of the Week



Average Departure Delay by Airline



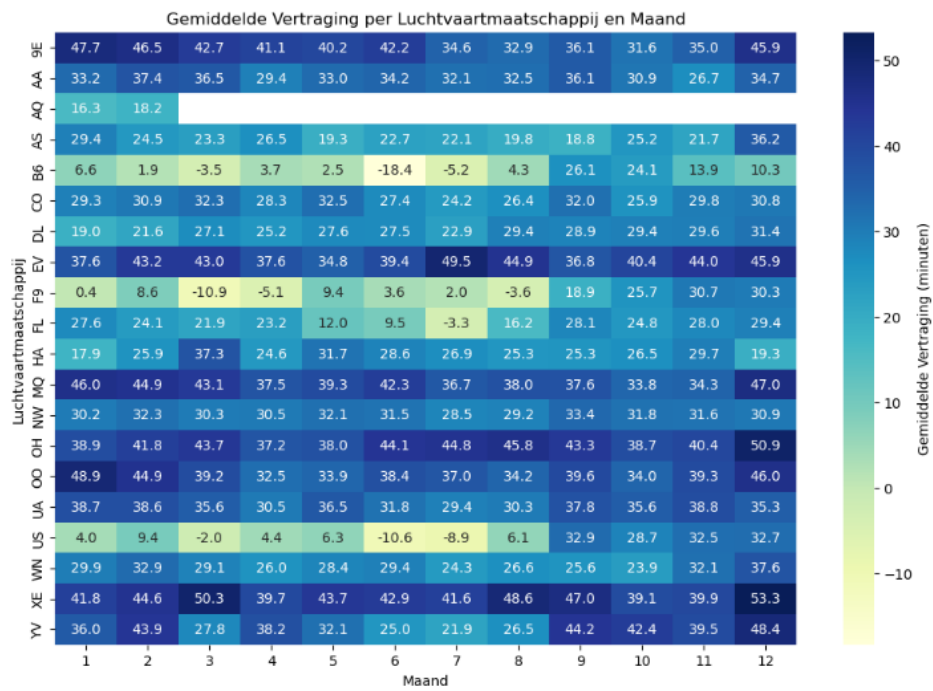
Departure Delays by Time of Day



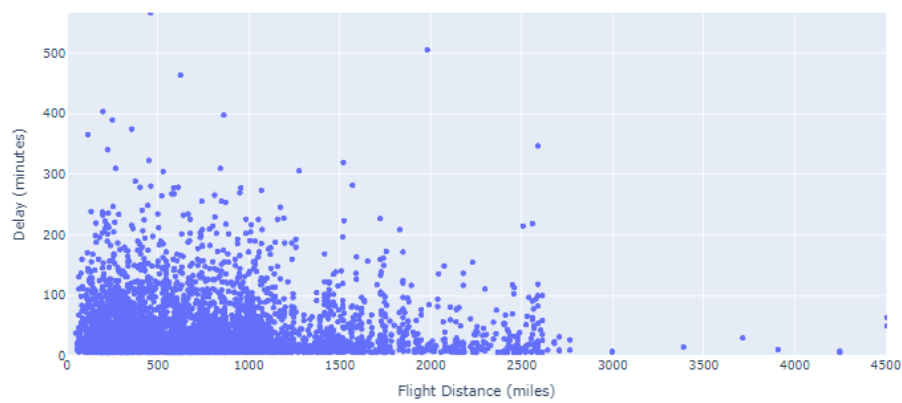
```

1 import seaborn as sns
2 import matplotlib.pyplot as plt
3
4 #bereken de gemiddelde vertraging per Luchtvaartmaatschappij en maand
5 heatmap_data = dfData.groupby(['UniqueCarrier', 'Month'])['DepartureDifference'].mean().unstack()
6
7 plt.figure(figsize=(12, 8))
8 sns.heatmap(heatmap_data, annot=True, fmt=".1f", cmap="YlGnBu", cbar_kws={'label': 'Gemiddelde Vertraging (minuten)'})
9 plt.title('Gemiddelde Vertraging per Luchtvaartmaatschappij en Maand')
10 plt.xlabel('Maand')
11 plt.ylabel('Luchtvaartmaatschappij')
12 plt.show()
13
14 import plotly.express as px
15
16 #sample 5000 rows from the dataset for performance
17 sample_data = dfData.sample(n=5000, random_state=1)
18
19 fig = px.scatter(
20     sample_data,
21     x='Distance',
22     y='DepartureDifference',
23     title="Flight Distance vs. Delay",
24     labels={'Distance': 'Flight Distance (miles)', 'DepartureDifference': 'Delay (minutes)'}
25 )
26
27 # Update Layout to set the range for both axes
28 fig.update_layout(
29     xaxis=dict(range=[0, sample_data['Distance'].max()], title='Flight Distance (miles)'),
30     yaxis=dict(range=[0, sample_data['DepartureDifference'].max()], title='Delay (minutes)')
31 )
32
33 fig.show()
34
35 #summing each delay cause across all flights
36 delay_causes = dfData[['CarrierDelay', 'WeatherDelay', 'NASDelay', 'SecurityDelay', 'LateAircraftDelay']].sum()
37
38 #plot
39 fig = px.pie(values=delay_causes, names=delay_causes.index, title="Distribution of Delay Causes")
40 fig.update_traces(textposition='inside', textinfo='percent+label')
41
42 #show figure
43 fig.show()
44
45 #calculate cancellation rate by airline
46 cancellation_rate = dfData.groupby('UniqueCarrier')['Cancelled'].mean().reset_index()
47
48 #plot
49 fig = px.bar(cancellation_rate, x='UniqueCarrier', y='Cancelled',
50             title='Cancellation Rate by Airline',
51             labels={'UniqueCarrier': 'Airline', 'Cancelled': 'Cancellation Rate'})
52
53 #show figure
54 fig.show()

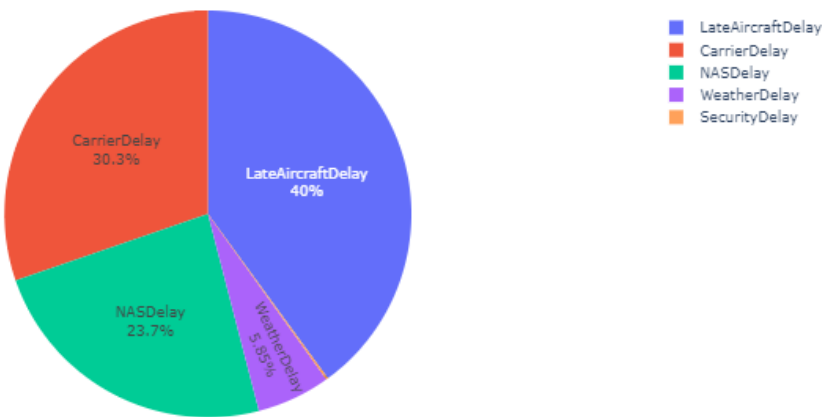
```



Flight Distance vs. Delay



Distribution of Delay Causes



Cancellation Rate by Airline

